Heterogeneous Graph Modeling for Anti-Money Laundering: A Reproducible Study on Elliptic++

Abdullah Almekhyal, Yousef Joudeh
Computer Science Department, College of Science
Kuwait University, Kuwait
s2231122390@ku.edu.kw, s22221110617@ku.edu.kw

***Abstract*—This paper applies a heterogeneous graph neural network approach to anti-money laundering detection on the publicly available Elliptic++ Bitcoin dataset. Inspired by the HMPNN framework of Johannessen and Jullum, we construct a heterogeneous graph with two node types (wallets and transactions) and four edge types, and train a model using PyTorch Geometric. The model achieves a PR-AUC of 0.8289, ROC-AUC of 0.9718, and 92% recall on the illicit class, demonstrating that heterogeneous graph modeling is effective for AML detection on a reproducible public benchmark.**

I. Introduction

Financial crimes, particularly money laundering, remain a critical challenge for financial institutions and regulatory authorities worldwide. In recent years, the Middle East region—and Kuwait specifically—has faced increasing pressure to strengthen Anti-Money Laundering (AML) frameworks, especially following its inclusion in the Financial Action Task Force (FATF) gray list. This classification highlights the urgency of adopting advanced technological solutions to enhance detection capabilities and ensure compliance with international standards.

Traditional rule-based AML systems often struggle to detect complex and evolving laundering patterns, such as layering, structuring, and smurfing. These systems rely heavily on predefined rules and fixed thresholds, making them less effective in identifying sophisticated and hidden relationships between entities. As financial crimes become more coordinated and structured, there is a growing need for intelligent approaches that can capture these hidden patterns.

Machine Learning (ML) has shown promising results in detecting suspicious financial activities. However, many existing approaches treat transactions independently and fail to capture the relational nature of financial systems. This limitation reduces their effectiveness in identifying coordinated laundering schemes that span multiple accounts and transactions. Graph Neural Networks (GNNs) address this limitation by operating directly on network-structured data, learning from both entity features and the relationships between entities simultaneously.

Recent work by Johannessen and Jullum [9] demonstrated the effectiveness of heterogeneous GNNs for AML detection on a large real-world banking graph from DNB, Norway's largest bank. Their Heterogeneous Message Passing Neural Network (HMPNN) achieved strong results by leveraging multiple node types and edge types. However, their model was evaluated on a private dataset that is not publicly available, limiting reproducibility and comparison by other researchers.

In this work, we address this limitation by applying a heterogeneous graph-based approach for AML detection to the Elliptic++ dataset [7], a publicly available Bitcoin transaction and wallet address dataset. We construct a heterogeneous graph with two node types (wallets and transactions) and four edge types, and evaluate the proposed approach using metrics suitable for highly imbalanced AML data. By transferring the heterogeneous modeling paradigm from a private banking dataset to a public benchmark, this work contributes toward reproducible and comparable AML detection research applicable to financial institutions in Kuwait and the broader region.

II. Related Work

A. Synthetic Banking Data and AML Graph Benchmarks

Weber *et al.* introduced AMLSim together with an early graph-learning study on a large synthetic banking network, positioning AML as a graph problem with massive, dense, and dynamic transaction structure [1]. Their work established both the simulator and a first GCN/FastGCN scalability benchmark. However, the study was preliminary and focused more on computational feasibility than on richer relational modeling.

Silva *et al.* provided one of the most directly comparable AMLSim studies by evaluating GCN, Skip-GCN, and a Node-Edge Neural Network (NENN) under several class-imbalance ratios [2]. Their results showed that explicitly incorporating edge information and using XGBoost on learned embeddings can outperform simpler graph classifiers, especially for the illicit class.

Jensen *et al.* introduced SynthAML, a privacy-preserving synthetic AML benchmark derived from real banking alerts and transactions [3]. Although SynthAML is not a graph dataset, it highlights the need for public AML benchmarks that preserve operational realism while enabling reproducible comparisons.

Li *et al.* studied AML in the setting of central bank digital currency and used AMLSim as a proxy for a "real-name" banking scenario [4]. Their results showed that Random Forest and XGBoost remained competitive or stronger than graph methods in the real-name scenario, reinforcing the need to compare graph methods against strong tabular baselines.

B. Public Graph-Learning Benchmarks for AML

Weber *et al.* released the Elliptic benchmark and compared classical machine learning models with several GNN variants on illicit Bitcoin transaction detection [5]. The study demonstrated that graph structure can improve AML detection, but also showed that strong non-graph methods such as Random Forest remain difficult baselines to beat.

Lo *et al.* proposed Inspection-L, a two-stage self-supervised framework that combines Deep Graph Infomax with a GIN encoder and a downstream Random Forest classifier [6]. Their results on Elliptic showed that self-supervised embeddings plus raw transactional attributes can outperform earlier supervised baselines.

Elmougy and Liu introduced Elliptic++, which extends the original Elliptic data into multiple graph views, including transaction-to-transaction, address-to-address, and address-to-transaction graphs [7]. They showed that richer graph construction and feature augmentation can improve illicit transaction and illicit address detection. This dataset is directly adopted in our work as the basis for heterogeneous graph construction.

C. Real-World Banking AML and Heterogeneous Relational Modeling

Cardoso *et al.* proposed LaundroGraph, a fully self-supervised graph representation learning approach for banking AML review support using a customer-transaction bipartite graph [8]. The paper showed that richer relational structure can improve both classification and analyst-assistance tasks.

Johannessen and Jullum proposed a heterogeneous message passing neural network (HMPNN) for suspicious customer detection on a large real-world banking graph containing multiple node and edge types [9]. Their results demonstrated that heterogeneous message passing can outperform classical machine learning and other graph baselines under severe class imbalance. This is the most relevant anchor paper for the current project because it directly motivates transferring heterogeneous AML modeling from a private banking dataset to a reproducible public benchmark.

Taken together, these studies reveal a clear progression in AML research: from early scalable graph experiments on synthetic data [1], to stronger baselines with edge-aware learning [2], to public crypto benchmarks with self-supervised and multi-view graph learning [5], [6], [7], and finally to heterogeneous real-banking models [8], [9]. What is still missing is a public benchmark study that brings heterogeneous graph modeling to the Elliptic++ dataset and evaluates it rigorously against both homogeneous GNNs and competitive tabular models.

III. Novelty Statement

The chosen paper by Johannessen and Jullum demonstrates that heterogeneous message passing can improve anti-money laundering detection; however, their model is evaluated on a private banking dataset that is not publicly available, which limits reproducibility and comparison by other researchers. In our work, we address this limitation by using Elliptic++ [7], a publicly available dataset of Bitcoin transactions and wallet addresses that naturally supports heterogeneous graph construction with multiple node and edge types.

This enables a reproducible experimental setup where the heterogeneous modeling approach can be tested under controlled conditions. We extend the original approach by constructing a heterogeneous graph from Elliptic++ with two node types (wallets and transactions) and four directed edge types, and evaluating its performance against established baselines. In addition, we report key metrics such as PR-AUC, ROC-AUC, F1-score, and recall for the suspicious class, providing a clear assessment of whether heterogeneous modeling improves AML detection on a public benchmark.

IV. Methodology

This section presents the proposed heterogeneous graph-based approach for detecting suspicious financial activities using the Elliptic++ dataset. The methodology includes the original paper's approach, our extension, graph construction, preprocessing, and the evaluation framework.

A. Original Approach: HMPNN

Johannessen and Jullum [9] proposed the Heterogeneous Message Passing Neural Network (HMPNN), which extends the homogeneous MPNN framework of Gilmer *et al.* [10] to operate on heterogeneous graphs. The key idea is to perform separate message passing operations for each *meta-step*—each unique combination of source node type, edge type, and target node type—using distinct learned message and update functions. This allows the model to capture type-specific interaction patterns.

The HMPNN algorithm operates in K iterations. At each iteration, for every node v of type ν, and for each meta-step $s = (\mu, \epsilon, \nu)$ ending at v, the model computes:

$$m_v^{(\mu,\epsilon,k)} = \sum_{u \in N_\mu^\epsilon(v)} M_k^{(\mu,\epsilon,\nu)}(h_v^{(k-1)}, h_u^{(k-1)}, r_{uv}^\epsilon) \quad (1)$$

$$h_v^{(\mu,\epsilon,k)} = U_k^{(\mu,\epsilon,\nu)}(h_v^{(k-1)}, m_v^{(\mu,\epsilon,k)}) \quad (2)$$

The intermediate representations from all meta-steps are then aggregated into a single node representation using a learned aggregation function $A_k^{(\nu)}$. Two aggregation variants were proposed: HMPNN-sum (element-wise summation) and HMPNN-ct (concatenation followed by a single-layer neural network).

The original model was evaluated on a private graph from DNB bank containing 5.1 million nodes (individuals, organizations, externals) and 10.2 million edges (transactions, business roles), with 9 distinct meta-steps. The task was to classify individual customers as suspicious or regular.

B. Our Extension: Heterogeneous GNN on Elliptic++

We adopt the same principle of heterogeneous graph modeling and apply it to the Elliptic++ dataset [7], a publicly available Bitcoin blockchain dataset. This transfer serves two

purposes: (1) it enables reproducibility, since the original DNB data is private, and (2) it tests whether heterogeneous modeling generalizes beyond traditional banking to cryptocurrency transaction networks.

Our model uses a heterogeneous graph convolution architecture implemented with PyTorch Geometric's `HeteroConv` module, inspired by the HMPNN-sum variant from the original paper. For each edge type, we apply a separate `SAGEConv` [14] layer, and the resulting node embeddings from different edge types are aggregated via summation. Note that unlike the original HMPNN, which uses a learned message function conditioned on edge features, our implementation uses SAGEConv operators that do not incorporate explicit edge features. This design choice reflects the structure of the Elliptic++ dataset, where relational information is captured through distinct edge types rather than continuous edge attributes. The model consists of two message-passing layers with ReLU activation, dropout regularization, and a final linear classification head.

Table I summarizes the key differences between our study and the anchor paper.

TABLE I
COMPARISON: OUR STUDY VS. ANCHOR PAPER

Aspect	Ours (Elliptic++)	Anchor (DNB)
Dataset	Public	Private
Domain	Bitcoin	Banking
Node types	2	3
Edge types	4	2 (9 meta-steps)
Total nodes	1,472,029	5,149,159
Total edges	4,417,560	10,202,950
Task	Wallet classif.	Individual classif.
Train/Test split	80/20 stratified	80/20 stratified
Class rebalancing	Yes (class-weighted)	No

C. Dataset: Elliptic++

The Elliptic++ dataset [7] is a publicly available Bitcoin blockchain dataset containing both transaction-level and wallet-level (actor-level) data. It extends the original Elliptic dataset [5] by adding wallet address information and multiple graph views.

1) Node Types and Features: Two node types are defined in our heterogeneous graph:

- **Wallet (Account) nodes**: 1,268,260 nodes representing Bitcoin wallet addresses, each with 56 features including aggregated transaction statistics such as total incoming/outgoing amounts, transaction counts, and temporal activity patterns.
- **Transaction nodes**: 203,769 nodes representing individual Bitcoin transactions, each with 182 features. The first 94 features capture local transaction information, while the remaining features represent aggregated neighborhood statistics. Features are anonymized for privacy.

Table II summarizes the node statistics.

2) Edge Types: Four directed edge types define the relationships in the graph:

TABLE II
NODE TYPES IN THE HETEROGENEOUS GRAPH

Type	#Nodes	#Features	#Labeled
Wallet (Account)	1,268,260	56	367,472
Transaction	203,769	182	46,564
Total	**1,472,029**	–	–

- **sends** (wallet → transaction): a wallet initiates a transaction (477,117 edges)
- **received_by** (transaction → wallet): a transaction delivers funds to a wallet (837,124 edges)
- **interacts_with** (wallet → wallet): direct interaction between wallets (2,868,964 edges)
- **flows_to** (transaction → transaction): money flow between transactions (234,355 edges)

Table III provides the full edge statistics.

TABLE III
EDGE TYPES (META-STEPS) IN THE HETEROGENEOUS GRAPH

Edge Type	#Edges
wallet → txn (sends)	477,117
txn → wallet (received_by)	837,124
wallet → wallet (interacts_with)	2,868,964
txn → txn (flows_to)	234,355
Total	**4,417,560**

3) Labels and Class Imbalance: Wallet nodes are labeled as illicit (class 1), licit (class 0), or unknown. Among all labeled wallets, 14,266 are illicit and 251,088 are licit, yielding an overall illicit ratio of approximately 5.38%. However, because a large portion of wallet nodes are unlabeled, the effective illicit ratio among the labeled subset used for training and evaluation is 7.78%. This severe class imbalance is characteristic of real-world AML data—the anchor paper reported an even more extreme ratio of 0.45%.

D. Preprocessing

The preprocessing pipeline consists of the following steps:

1) **Data loading**: All CSV files from the Elliptic++ dataset are loaded, including node features, node labels, and edge lists for all four relation types.
2) **Missing value handling**: 16,405 NaN values identified in transaction features are replaced with zeros.
3) **Feature normalization**: Features are normalized per node type using StandardScaler (zero mean, unit variance), applied separately to wallet and transaction features.
4) **Node ID mapping**: Wallet addresses and transaction IDs are remapped to contiguous integer indices for compatibility with PyTorch Geometric.
5) **Graph construction**: A PyTorch Geometric `HeteroData` object is assembled with all node features, labels, and edge indices.

E. Experimental Setup

The primary prediction task is to classify wallet nodes (accounts) as illicit or licit based on their features and their

structural relationships in the heterogeneous graph. This mirrors the anchor paper's task of classifying individual bank customers.

1) Train/Test Split: Following the anchor paper, we use an 80/20 stratified random split on labeled wallet nodes. The stratification preserves the class balance across both sets, resulting in a 7.78% illicit ratio in both train and test sets. Unlabeled nodes remain in the graph during training and inference to support message passing, consistent with the semi-supervised setting of the anchor paper.

Table IV shows the resulting split statistics.

TABLE IV
TRAIN/TEST SPLIT FOR WALLET NODES

Set	Illicit	Licit	Illicit Ratio
Train (80%)	22,881	271,096	7.78%
Test (20%)	5,720	67,775	7.78%

2) Model Architecture: The heterogeneous GNN model uses two message-passing layers. Each layer applies a separate SAGEConv operator per edge type, and aggregates the resulting embeddings via summation, followed by ReLU activation and dropout regularization. A per-type linear classification head produces the final illicit/licit prediction. To address the class imbalance, we apply inverse-frequency class weighting in the cross-entropy loss function. Full hyperparameter details are provided in Section V-B.

F. Evaluation Metrics

Due to the highly imbalanced nature of AML data, we use multiple evaluation metrics:

- **PR-AUC** (Area under Precision-Recall curve): the primary metric, following the anchor paper, due to its suitability for severe class imbalance.
- **ROC-AUC** (Area under Receiver Operating Characteristic curve): a widely used threshold-independent measure.
- **Precision at fixed recall levels** (1%, 5%, 10%, 50%): directly interpretable for operational AML deployment, following the anchor paper's reporting format.
- **F1-score**: harmonic mean of precision and recall for the illicit class.
- **Classification report**: per-class precision, recall, and F1.

G. System Overview

Table V summarizes the overall pipeline of the proposed system.

TABLE V
SYSTEM OVERVIEW

Step	Description
Dataset	Elliptic++ (public, Bitcoin blockchain)
Preprocessing	NaN handling, normalization, ID mapping
Graph Construction	Wallet + Transaction nodes (heterogeneous)
Relations	4 edge types: sends, received_by, interacts_with, flows_to
Modeling	Heterogeneous GNN (HeteroConv with SAGEConv)
Task	Wallet (account) classification: illicit vs. licit
Evaluation	PR-AUC, ROC-AUC, Precision@Recall, F1

V. EXPERIMENTS

A. Implementation Environment

All experiments were implemented in Python 3.10 using PyTorch [11] and PyTorch Geometric (PyG) [12] for heterogeneous graph operations.[1] Sparse message-passing kernels were provided by `torch_scatter` and `torch_sparse`. Evaluation metrics were computed with `scikit-learn` [13]. Training was conducted on Google Colaboratory using an NVIDIA T4 GPU (16 GB VRAM) with CUDA 12.1.

B. Model Configuration

We implement a heterogeneous GNN inspired by the HMPNN framework proposed by Johannessen and Jullum [9]. The model consists of two stacked `HeteroConv` layers, each applying `SAGEConv` [14] independently per relation type with sum-aggregation across relations, followed by a per-type linear classifier head. This corresponds to the HMPNN-sum aggregation variant, adapted to use SAGEConv operators in place of the original edge-feature-conditioned message functions. Table VI lists the hyperparameters used.

TABLE VI
MODEL AND TRAINING HYPERPARAMETERS.

Hyperparameter	Value
Hidden channels	64
Conv layers	2
Dropout	0.3
Optimiser	Adam
Learning rate	0.005
Weight decay	10^{-4}
Loss	Cross-Entropy (class-weighted)
Epochs	100

C. Results

Table VII reports the final test-set performance after 100 training epochs on the wallet account classification task.

TABLE VII
FINAL TEST-SET RESULTS — WALLET ACCOUNT CLASSIFICATION (ELLIPTIC++).

Metric	Score
ROC-AUC	0.9718
PR-AUC	0.8289
F1 (weighted)	0.9183
Accuracy	0.9037
P@R1%	1.0000
P@R5%	0.9995
P@R10%	0.9952
P@R50%	0.9445
Precision — Licit (0)	0.99
Recall — Licit (0)	0.90
F1 — Licit (0)	0.95
Precision — Illicit (1)	0.44
Recall — Illicit (1)	0.92
F1 — Illicit (1)	0.60

[1] Code available at: https://github.com/abojoudah/HMPNN-MLD-EllipticDataset

The confusion matrix on the 73,495 test wallet nodes is:

$$C = \begin{pmatrix} 61{,}127 & 6{,}648 \\ 430 & 5{,}290 \end{pmatrix}$$

where rows represent true classes (licit, illicit) and columns represent predicted classes. Of the 5,720 illicit wallets in the test set, the model correctly identifies 5,290 (92% recall), missing only 430.

The Precision@Recall results are particularly strong: at only 1% recall the model achieves perfect precision (1.0), and even at 50% recall it maintains precision of 0.94 — indicating that when the model flags a wallet as suspicious, it is highly reliable. The lower precision of 0.44 at the default threshold reflects the severe class imbalance (7.78% illicit) rather than model weakness.

Table VIII shows metric progression across training epochs.

TABLE VIII
TRAINING PROGRESSION — WALLET CLASSIFICATION (SELECTED EPOCHS).

Epoch	Loss	F1	PR-AUC	ROC-AUC
1	0.7251	0.9020	0.3349	0.8423
10	0.4223	0.8658	0.4845	0.9054
20	0.3606	0.8567	0.6336	0.9319
30	0.3301	0.8637	0.6808	0.9419
40	0.3036	0.8819	0.7181	0.9505
50	0.2851	0.8925	0.7432	0.9561
60	0.2671	0.8970	0.7699	0.9609
70	0.2535	0.9008	0.7894	0.9649
80	0.2412	0.9078	0.8074	0.9679
90	0.2331	0.9114	0.8196	0.9701
100	0.2200	0.9183	0.8289	0.9718

To contextualize these results, Table IX compares our findings against those reported by Johannessen and Jullum [9] on the private DNB banking dataset. While a direct comparison is limited by the fundamental differences in dataset domain (Bitcoin vs. banking), graph scale, class imbalance severity, and labeling definitions, the table highlights how the heterogeneous modeling paradigm performs across different financial contexts.

TABLE IX
COMPARISON OF RESULTS: OUR STUDY (ELLIPTIC++) VS. ANCHOR PAPER (DNB). THE ANCHOR PAPER'S BEST MODEL (HMPNN-CT, 2 HIDDEN LAYERS) IS USED FOR COMPARISON.

Metric	Ours (Elliptic++)	Anchor (DNB)
ROC-AUC	0.9718	0.9083
PR-AUC	0.8289	0.1800
P@R1%	1.0000	0.6667
P@R5%	0.9995	0.5821
P@R10%	0.9952	0.5099
P@R50%	0.9445	0.1025
Illicit ratio	7.78%	0.45%
Dataset size	1.47M nodes	5.15M nodes
Graph type	Bitcoin	Banking

Our model achieves substantially higher PR-AUC (0.8289 vs. 0.1800) and Precision@Recall scores across all thresholds. However, this difference is largely explained by the much more severe class imbalance in the anchor paper's dataset (0.45% suspicious vs. our 7.78% illicit). With a larger proportion of positive examples, our model has significantly more illicit behavior to learn from during training, making the classification task inherently easier. The ROC-AUC values are more comparable (0.9718 vs. 0.9083), as this metric is less sensitive to class imbalance.

D. Issues Encountered

a) Label dtype mismatch.: Label tensors loaded from the `.pt` file were not guaranteed to be one-dimensional integer tensors, causing `torch.bincount` and `CrossEntropyLoss` to raise runtime errors. This was resolved by applying `.long().view(-1)` to all label tensors immediately after loading.

b) Unlabelled nodes.: The Elliptic++ dataset assigns label -1 to nodes whose ground-truth class is unknown. These nodes are retained in the graph for message passing but excluded from the loss and evaluation by intersecting the pre-existing train/test masks with the condition $y \geq 0$.

c) Class imbalance.: With only 7.78% of labelled wallet nodes being illicit, a naive model trivially achieves high accuracy by predicting all wallets as licit. Inverse-frequency class weights in the loss function substantially improved illicit-class recall from near zero to 0.92.

VI. DISCUSSION

A. Interpretation of Results

The heterogeneous GNN model achieves strong detection performance on the Elliptic++ dataset, with a ROC-AUC of 0.9718 and PR-AUC of 0.8289. The model correctly identifies 92% of illicit wallets while maintaining perfect precision at low recall thresholds (P@R1% = 1.0). These results confirm the central claim of Johannessen and Jullum [9]: that heterogeneous graph modeling is effective for anti-money laundering detection.

The Precision@Recall metrics are particularly relevant for operational AML deployment. At 1% recall, the model flags a small number of wallets with perfect precision, meaning every flagged wallet is truly illicit. Even at 50% recall, precision remains above 94%. This is directly useful for financial institutions with limited investigation resources, as it allows them to prioritize the most confident predictions.

The training progression in Table VIII shows steady improvement across all metrics over 100 epochs, with PR-AUC rising from 0.3349 to 0.8289. The loss continues to decrease at epoch 100, suggesting that additional training epochs or hyperparameter tuning could yield further gains.

B. Comparison with the Anchor Paper

While our model achieves higher absolute scores than the anchor paper's best model (HMPNN-ct), a direct comparison is not straightforward for several reasons. First, the datasets differ fundamentally in domain (cryptocurrency vs. traditional banking), scale (1.47M vs. 5.15M nodes), and class imbalance (7.78% vs. 0.45% positive rate). Second, our implementation uses SAGEConv operators without edge features, whereas the

original HMPNN uses learned message functions conditioned on edge features such as transaction amounts and counts. Third, we apply class-weighted loss to address imbalance, while the anchor paper preserved the natural class distribution during training.

Despite these differences, both studies demonstrate the same core finding: heterogeneous graph neural networks that model multiple node and edge types are effective for AML detection. The fact that this approach works on both a private banking graph with three node types and nine meta-steps, and a public Bitcoin graph with two node types and four edge types, suggests that the heterogeneous modeling paradigm generalizes across financial domains.

C. Limitations

This work has several limitations. First, our comparison with the anchor paper is constrained by differences in dataset domain, scale, and class imbalance, which makes direct performance comparison difficult. A more rigorous evaluation would include non-graph baselines trained on the same Elliptic++ data to isolate the contribution of the graph structure.

Second, our implementation approximates the HMPNN architecture using SAGEConv rather than implementing the full edge-feature-conditioned message passing. This means our model does not leverage continuous edge attributes such as transaction amounts, which were shown to be important in the anchor paper.

Third, the Elliptic++ dataset, while publicly available, represents Bitcoin transactions with anonymized features. The transferability of these results to traditional banking systems, which have richer and more interpretable customer and transaction features, remains an open question.

Finally, we did not explore the HMPNN-ct aggregation variant, which uses concatenation followed by a single-layer neural network and was the best-performing model in the anchor paper. Our sum-aggregation approach corresponds to the simpler HMPNN-sum variant.

D. Future Work

Several directions could extend this work. First, implementing the full HMPNN architecture with edge-feature-conditioned message functions would provide a more faithful reproduction of the anchor paper's approach and could improve performance. Second, adding non-graph baselines (Random Forest, XGBoost) trained on wallet features would clarify the added value of the graph structure. Third, exploring the HMPNN-ct aggregation variant and comparing it against HMPNN-sum on the same dataset could reveal whether the concatenation-based approach offers similar advantages on Elliptic++ as it did on the DNB dataset. Finally, incorporating temporal information from the Elliptic++ dataset could enable dynamic graph modeling, capturing how suspicious patterns evolve over time.

VII. Conclusion

This paper applied a heterogeneous graph neural network approach, inspired by the HMPNN framework of Johannessen and Jullum [9], to the publicly available Elliptic++ Bitcoin dataset for anti-money laundering detection. We constructed a heterogeneous graph with two node types (wallets and transactions) and four directed edge types, and trained a model using PyTorch Geometric's HeteroConv module with SAGEConv operators and sum-aggregation across relation types.

The model achieved a PR-AUC of 0.8289, ROC-AUC of 0.9718, and 92% recall on the illicit class, demonstrating strong detection performance under class imbalance. The Precision@Recall analysis showed that the model produces highly reliable predictions at low recall thresholds, which is directly relevant for operational AML systems where investigation capacity is limited. While direct comparison with the anchor paper is complicated by differences in dataset domain, scale, and class imbalance, both studies support the same conclusion: modeling multiple node and edge types through heterogeneous graph neural networks is an effective approach for detecting suspicious financial activity. By transferring this paradigm to a public benchmark, this work contributes a reproducible evaluation that can serve as a foundation for future AML research.

References

[1] M. Weber, J. Chen, T. Suzumura, A. Pareja, T. Ma, H. Kanezashi, T. Kaler, C. E. Leiserson, and T. B. Schardl, "Scalable Graph Learning for Anti-Money Laundering: A First Look," *arXiv:1812.00076*, 2018.

[2] Í. D. G. Silva *et al.*, "Graph Neural Networks Applied to Money Laundering Detection," in *Proc. XIX Brazilian Symposium on Information Systems*, 2023, doi: 10.1145/3592813.3592912.

[3] R. I. T. Jensen *et al.*, "A Synthetic Data Set to Benchmark Anti-Money Laundering Methods," *Scientific Data*, vol. 10, no. 1, p. 661, 2023, doi: 10.1038/s41597-023-02569-2.

[4] Z. Li, Y. Zhang, Q. Wang, and S. Chen, "Transactional Network Analysis and Money Laundering Behavior Identification of Central Bank Digital Currency of China," *Journal of Social Computing*, vol. 3, no. 3, pp. 219–230, 2022, doi: 10.23919/JSC.2022.0011.

[5] M. Weber *et al.*, "Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics," *arXiv:1908.02591*, 2019.

[6] W. W. Lo, M. Sarhan, S. Layeghy, and M. Portmann, "Inspection-L: Self-Supervised GNN-Based Money Laundering Detection System for Bitcoin," *arXiv:2203.10465*, 2022.

[7] Y. Elmougy and Y. Liu, "Demystifying Fraudulent Transactions and Illicit Nodes in the Bitcoin Network for Financial Forensics," in *Proc. 29th ACM SIGKDD*, 2023, doi: 10.1145/3580305.3599803.

[8] M. Cardoso, P. Saleiro, and P. Bizarro, "LaundroGraph: Self-Supervised Graph Representation Learning for Anti-Money Laundering," in *Proc. 3rd ACM Intl. Conf. on AI in Finance*, 2022, doi: 10.1145/3533271.3561727.

[9] F. Johannessen and M. Jullum, "Finding Money Launderers Using Heterogeneous Graph Neural Networks," *Journal of Finance and Data Science*, vol. 11, 2025.

[10] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural Message Passing for Quantum Chemistry," in *Proc. 34th International Conference on Machine Learning (ICML)*, PMLR, pp. 1263–1272, 2017.

[11] A. Paszke *et al.*, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[12] M. Fey and J. E. Lenssen, "Fast Graph Representation Learning with PyTorch Geometric," in *Proc. ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[13] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[14] W. L. Hamilton, Z. Ying, and J. Leskovec, "Inductive Representation Learning on Large Graphs," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.